CRCA-Context: Counterfactual Robustness of Repository Context Retrieval Under Equivalent Issue Descriptions

Anonymous Authors[1]

Abstract

Before a repository-level coding agent edits code, it must identify which project files are relevant to the issue. Current evaluations usually assess this context-selection step from a single issue statement, although users describe the same bug in many equivalent ways. We introduce CRCA-Context, an open benchmark requiring no paid API calls for counterfactual robustness in repository context retrieval. For each SWE-bench Lite issue, we construct deterministic semantics-preserving views through compression, bulletization, user-style wrapping, nuisance instructions, and distractor context. Using changed files from gold patches as proxy file-relevance labels, we evaluate BM25, TF-IDF, and identifier-based retrieval under these views. Proxy documents exclude added patch lines to avoid solution leakage. On all 300 SWE-bench Lite tasks, BM25 falls from 0.203 original F1@5 to 0.187 worst-view F1@5, while TF-IDF falls from 0.210 to 0.190. A deployment-mode Stable Context Selection baseline uses generated companion views from one observed input and slightly improves BM25 robust hit@5 from 0.560 to 0.567. A 50-task repository-corpus validation shows the same qualitative pattern under a harder retrieval setting. CRCA-Context isolates file-level retrieval as a lightweight diagnostic for that step in human-centered coding-agent workflows, alongside benchmarks that score end-to-end patch success.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Under review at the DL4C 2026 Workshop. Do not distribute.

1. Introduction

Repository-level coding-agent evaluations commonly start from one fixed issue statement, as in SWE-bench-style issue-resolution benchmarks (Jimenez et al., 2024). This is convenient, but it hides an ordinary interaction problem: users describe tasks in many equivalent surface forms. A developer may compress a bug report, paste it as bullets, add a harmless instruction such as "preserve public APIs," or include an unrelated note from the same discussion. Prior work on prompt variability and repository-level instruction files suggests that such surface context can affect code-model and coding-agent behavior (Paleyes et al., 2025; Gloaguen et al., 2026). The correct fix stays the same across those surface forms; ideally, so does the agent's retrieved repository context.

This paper targets repository context retrieval, the step that precedes patch generation in full pipelines. Full coding-agent evaluation depends on agent-computer interfaces and expensive execution loops (Yang et al., 2024). Context retrieval can be studied with public benchmark rows, lexical retrievers, and CPU-only scripts, and is already recognized as a central process-level bottleneck for coding agents (Li et al., 2026). This makes it a useful target for an open DL4C artifact while still addressing a coding-agent reliability question.

Our framing is invariance-inspired. The underlying software issue is held fixed; nuisance aspects of the presentation vary. A robust retrieval system should preserve task-relevant files across those equivalent views. We use "counterfactual" in the benchmark sense of controlled equivalent input views, while remaining agnostic about internal causal mechanisms in a model. CRCA-Context operationalizes this idea with deterministic interventions, cross-view retrieval metrics, and a simple stability-based mitigation.

2. Benchmark Construction

We use all 300 tasks in SWE-bench Lite (Jimenez et al., 2024). To keep the benchmark open and runnable

without human gold-context annotations, paid models, or official Docker execution, we parse each gold patch and use changed files as proxy relevant context. Relative to human-labeled context, some changed files are implementation targets rather than necessary evidence, and some useful files never appear in the patch. The label set is still a reproducible proxy for issue-local file relevance.

For each task, CRCA-Context creates six views. The original view is the unmodified problem statement. The compressed view shortens the issue while preserving technical identifiers. The bulletized view restructures the issue into bullets. The noisy-user view wraps the issue in informal developer language. The nuisance-instruction view appends generic repository guidance meant to leave the underlying task unchanged. The distractor-context view appends a plausible but irrelevant note. These transformations are deterministic and require no LLM calls.

We include a basic intervention audit. It checks whether extracted technical identifiers disappear and whether the transformed issue is too short or too long. Across the 1,800 generated views, 97.6% pass this audit. We also spot-checked 50 transformed views, stratified across the five non-original intervention types; all 50 preserved the target issue in this check. We retain flagged views in the main evaluation, but report the audit rate and release both audit files so readers can inspect transformation quality. The remaining automatic flags mostly involve compressed or bulletized views with complex examples; we keep them visible rather than treating semantic preservation as automatic.

The main experiment uses a lightweight proxy corpus. For each task, gold documents are pre-change hunk-context snippets from its changed files, and distractor documents are snippets sampled from other tasks. Proxy documents contain file paths, hunk headers, unchanged context lines, and removed lines; added lines from the gold patch are excluded to avoid solution leakage. This gives a scalable controlled changed-file retrieval stress test over all 300 tasks. To check whether the same pattern appears in a less controlled setting, we also run a smaller repository-corpus validation on 50 SWE-bench Lite tasks from shallow GitHub checkouts, with 60,854 indexed source and documentation files. The validation is harder and smaller, so we treat it as a qualitative check rather than the main benchmark.

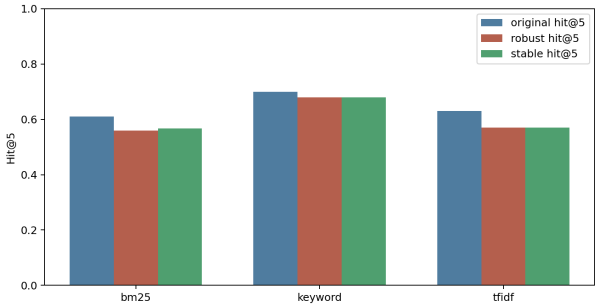

Figure 1. Original hit@5, robust hit@5, and deployment-mode Stable Context Selection hit@5. Robust hit requires retrieving a gold file under every issue view. Stable Context Selection is conservative here: it slightly improves BM25 and leaves TF-IDF and keyword retrieval unchanged.

3. Evaluation

We evaluate three free retrievers: BM25 (Robertson & Zaragoza, 2009), TF-IDF cosine retrieval, and keyword retrieval over technical identifiers. For each task, view, retriever, and $k \in \{1, 3, 5, 10\}$, the system returns top-k files. View-level metrics include F1@k, hit@k, and MRR. Cross-view metrics include original F1, mean F1, worst-view F1, counterfactual sensitivity, Jaccard stability, robust hit, and gold-file fragility. Sensitivity is max-view F1 minus min-view F1. Jaccard is the average pairwise overlap among top-k retrieved file sets across views. Robust hit requires at least one gold file in the top-k list for every view. Fragility is the fraction of gold files found in the original view that are missed in at least one intervention view.

Stable Context Selection. We add one no-training mitigation. Given one observed issue view and a base retriever, we generate deterministic companion views, retrieve under those companions, and rank files by how consistently they appear across top-k lists, using rank as a tie-breaker. The intuition is simple: files that survive multiple equivalent presentations are less likely to be artifacts of wording. For evaluation, Stable Context Selection is run separately from each observed input view: given one view, it generates deterministic companion views from that string alone, retrieves under those companions, and returns one stable file list. Each run conditions on a single observed issue string; companion views come only from applying the published deterministic transforms to that string. Scoring uses only strings those transforms emit from the current input—the gold patch and other pre-stored benchmark views stay outside this retrieval stage.

Table 1. Main proxy-corpus retrieval robustness results on all 300 SWE-bench Lite tasks at $k = 5$. Stable Context Selection is reported as a deployment-mode cross-view selector for each base retriever.

Retriever	Orig. F1	Mean F1	Worst F1	Stable F1	Sens.	Jac.	Frag.	Robust Hit	Stable Hit
BM25	0.203	0.203	0.187	0.203	0.038	0.818	0.050	0.560	0.567
Keyword	0.233	0.232	0.227	0.232	0.009	0.878	0.020	0.680	0.680
TF-IDF	0.210	0.207	0.190	0.207	0.037	0.796	0.060	0.570	0.570

4. Main Results

Figure 1 and Table 1 give the main proxy-corpus results at $k = 5$. Single-view retrieval is optimistic for BM25 and TF-IDF. BM25's original F1@5 is 0.203, while its worst-view F1@5 is 0.187. TF-IDF drops from 0.210 to 0.190. These changes are moderate, but they affect robustness: BM25 robust hit@5 is 0.560 and TF-IDF robust hit@5 is 0.570.

Stable Context Selection has a modest effect in deployment mode. BM25 stable hit@5 rises from 0.560 to 0.567, while TF-IDF and keyword robust hit are unchanged. Stable F1@5 is close to the base retrievers: 0.203 for BM25 and 0.207 for TF-IDF. The result is useful mainly as a conservative baseline: cross-view consistency needs no training or paid models, even though it only partially addresses retrieval brittleness.

Keyword retrieval should be interpreted as a sanity-check lower-complexity baseline rather than evidence that lexical matching solves repository understanding. Its stability follows from the benchmark design: deterministic interventions preserve technical identifiers. This result shows that CRCA-Context can distinguish identifier-anchored robustness from sensitivity to surrounding natural language. It also motivates future interventions based on user reports that omit or corrupt identifiers. As a small check, we split tasks by the number of extracted technical identifiers. Identifier-light tasks have lower robust hit for all retrievers; for BM25, robust hit@5 is 0.487 on identifier-light tasks versus 0.635 on identifier-heavy tasks. This supports the interpretation that preserved identifiers help; robustness checks stay informative even when identifiers are abundant.

Table 2 breaks results down by view. Nuisance instructions are the weakest aggregate view at F1@5 0.207 versus 0.216 for the original view. Noisy-user phrasing and distractor context are close behind. Bulletization slightly improves performance, likely because it exposes technical terms in a cleaner structure. Interventions change aggregate scores in both directions; taken together they clarify which cues each retriever relies on.

Table 2. Aggregate retrieval performance by issue view at $k = 5$. Values average over BM25, TF-IDF, and keyword retrieval.

View	F1	Hit	MRR	Δ vs. Orig.
Original	0.216	0.647	0.506	0.000
Compressed	0.218	0.654	0.518	-0.003
Bulletized	0.224	0.672	0.538	-0.009
Noisy user	0.209	0.626	0.483	0.007
Nuisance instr.	0.207	0.622	0.484	0.008
Distractor ctx.	0.209	0.628	0.482	0.006

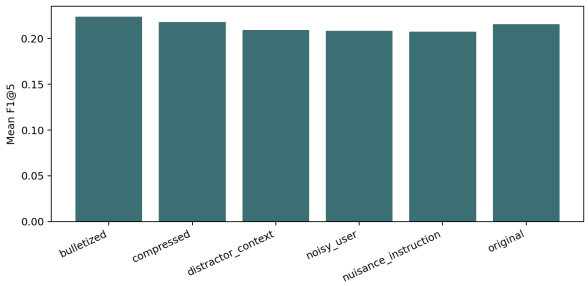

Figure 2. Aggregate F1@5 by intervention type. Nuisance instructions, distractor context, and noisy-user phrasing are the weakest views in the 300-task proxy experiment.

Table 3. Repository-corpus validation on 50 SWE-bench Lite tasks at $k = 5$. The validation indexes cloned repository files rather than the proxy corpus.

Retriever	Orig. F1	Worst F1	Sens.	Jac.	Robust Hit
BM25	0.107	0.073	0.040	0.669	0.220
Keyword	0.120	0.107	0.020	0.843	0.320
TF-IDF	0.053	0.040	0.040	0.661	0.120

5. Repository-Corpus Validation

The repository-corpus validation is smaller than the proxy experiment but important. On 50 tasks with cloned repository files, absolute retrieval scores are lower, as expected. The qualitative pattern remains: BM25 drops from 0.107 original F1@5 to 0.073 worst-view F1@5, while TF-IDF remains low and sensitive. Keyword retrieval is again more stable, with 0.120 original F1@5 and 0.107 worst-view F1@5. Scaling up repository-corpus retrieval remains future work; this

validation still cuts the risk that the main result is only an artifact of sampled patch snippets.

6. Limitations

CRCA-Context concentrates on context retrieval under equivalent issue views; patch generation, test execution, and full agent loops sit outside this benchmark. Changed files from SWE-bench patches serve as proxy relevance labels, and pre-change hunk-context snippets as proxy documents in the main experiment; strong retrieval here is a useful intermediate signal for patching, while patch correctness still calls for the usual execution-based checks. The proxy corpus encodes controlled changed-file retrieval; the 50-task repository-corpus validation asks whether the same qualitative pattern appears when indexing cloned repository files. We bundle free lexical retrievers for CPU-only runs; extending the released artifact with neural retrievers is a natural next step. Terminology follows invariance-inspired evaluation motivated by causal questions about robustness, while remaining agnostic about causal identification inside language models.

7. Conclusion

CRCA-Context turns a costly coding-agent robustness question into a runnable benchmark that needs no paid API calls. Across all 300 SWE-bench Lite tasks, single-view context retrieval hides brittleness for BM25 and TF-IDF; identifier-centered retrieval is more invariant under deterministic semantics-preserving views; and deployment-mode Stable Context Selection provides a conservative no-training baseline. More broadly, coding-agent evaluations should pair success on a single issue statement with stability of repository understanding when a human describes the same issue in different equivalent ways.

References

Gloaguen, T., Mündler, N., Müller, M., Raychev, V., and Vechev, M. Evaluating AGENTS.md: Are repository-level context files helpful for coding agents?, 2026. URL https://arxiv.org/abs/2602.11988. arXiv:2602.11988.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In International Conference on Learning Representations, 2024. URL https://arxiv.org/abs/2310.06770. arXiv:2310.06770.

Li, H., Zhu, L., Zhang, B., Feng, R., Wang, J., Pan, Y., Barr, E. T., Sarro, F., Chu, Z., and Ye, H. ContextBench: A benchmark for context retrieval in coding agents, 2026. URL https://arxiv.org/abs/2602.05892. arXiv:2602.05892.

Paleyes, A., Sendyka, R., Robinson, D., Cabrera, C., and Lawrence, N. D. Code roulette: How prompt variability affects LLM code generation, 2025. URL https://arxiv.org/abs/2506.10204. arXiv:2506.10204. Extended version of a paper accepted to LLM4Code at ICSE 2026.

Robertson, S. and Zaragoza, H. The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval, 3(4):333–389, 2009. doi: 10.1561/1500000019.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. In Advances in Neural Information Processing Systems, volume 37, 2024. URL https://arxiv.org/abs/2405.15793. arXiv:2405.15793.